Types of Programming Languages

Jim Hurst

Types of Programming Languages

Introduction

Computer science has made huge strides since it became recognized as a distinct academic discipline in the 1960s. One of the fundamental problems it has addressed is how to translate the problems that people need solved into a format that computers can process. A variety of powerful and ingenious solutions have been applied to this problem. Programming languages are human-engineered languages developed to convey instructions to machines; they are based on rules of syntax and semantics. Thousands of different programming languages have been developed, used, and discarded.

The struggle for survival among programming languages is ruthless. Languages arise, are found wanting, and are soon replaced by more clever or more flexible offspring.

Initially, the evolution of programming languages was driven by a quest for efficient translation of human language to machine code. This produced languages with high levels of abstraction, which hid the hardware and used representations that are convenient and comfortable to human programmers. At some point, another critical aspect of language design became managing the complexity of programs. As programs became larger and more sophisticated, developers realized that some language types were easier to support in large systems. This has lead to greater use of objected-oriented and event-driven programming languages.

History

One problem in discussing the history of computer science is where to start. Jacquard looms used early versions of punch cards, but they are not computing devices in any modern sense. Programming languages as understood today became possible when the work of Alan Turing and Alonzo Church in the 1930s and 40s provided formal mathematical means to express computational procedures. The first efforts in this direction consisted of painstakingly crafting instructions as strings of ones and zeros (or machine code). Humans do not excel at this type of logic, and these first generation languages were quickly replaced by a second generation of assembly languages.

Assembly languages use low-level mnemonics to model the underlying hardware and specify computations. In this sense, they are different than the other languages discussed here, because assembler code is a first level abstraction of the hardware. For example, a typical statement in assembler to command the processor to move the hexadecimal number 0x80 (decimal 128) into processor register r2 might be: mov r2, 080h

Although programming in assembly language is not as difficult and error prone as stringing together ones and zeros, it is slow and cumbersome. By the 1950s, modern general purpose programming languages had arrived to address this difficulty, including FORTRAN, LISP, and COBOL. These were known as third-generation languages.

Overview of Types

As might be expected in a dynamic and evolving field, there is no single standard for classifying programming languages. In fact, dozens of categories exist. One of the most fundamental ways programming languages are characterized is by programming

paradigm. A programming paradigm provides the programmer's view of code execution. The most influential paradigms are examined in the next three sections, in approximate chronological order. Each of these paradigms represents a mature worldview, with enormous amounts of research and effort expended in their development.

A given language is not limited to use of a single paradigm. Java, for example, supports elements of both procedural and object-oriented programming, and it can be used in a concurrent, event-driven way. Programming paradigms continue to grow and evolve, as new generations of hardware and software present new opportunities and challenges for software developers.

Procedural Programming Languages

Procedural programming specifies a list of operations that the program must complete to reach the desired state. This one of the simpler programming paradigms, where a program is represented much like a cookbook recipe. Each program has a starting state, a list of operations to complete, and an ending point. This approach is also known as imperative programming. Integral to the idea of procedural programming is the concept of a procedure call.

Procedures, also known as functions, subroutines, or methods, are small sections of code that perform a particular function. A procedure is effectively a list of computations to be carried out. Procedural programming can be compared to unstructured programming, where all of the code resides in a single large block. By splitting the programmatic tasks into small pieces, procedural programming allows a section of code to be re-used in the program without making multiple copies. It also makes it easier for programmers to understand and maintain program structure.

Two of the most popular procedural programming languages are FORTRAN and BASIC.

Structured Programming Languages

Structured programming is a special type of procedural programming. It provides additional tools to manage the problems that larger programs were creating. Structured programming requires that programmers break program structure into small pieces of code that are easily understood. It also frowns upon the use of global variables and instead uses variables local to each subroutine. One of the well known features of structural programming is that it does not allow the use of the GOTO statement. It is often associated with a "top-down" approach to design. The top-down approach begins with an initial overview of the system that contains minimal details about the different parts. Subsequent design iterations then add increasing detail to the components until the design is complete.

The most popular structured programming languages include C, Ada, and Pascal.

Object-Oriented Programming Languages

Object-oriented programming is one the newest and most powerful paradigms. In object-oriented programs, the designer specifies both the data structures and the types of operations that can be applied to those data structures. This pairing of a piece of data with the operations that can be performed on it is known as an object. A program thus

becomes a collection of cooperating objects, rather than a list of instructions. Objects can store state information and interact with other objects, but generally each object has a distinct, limited role.

There are several key concepts in object-oriented programming (OOP). A class is a template or prototype from which objects are created, so it describes a collection of variables and methods (which is what functions are called in OOP). These methods can be accessible to all other classes (public methods) or can have restricted access (private methods). New classes can be derived from a parent class. These derived classes inherit the attributes and behavior of the parent (inheritance), but they can also be extended with new data structures and methods.

The list of available methods of an object represents all the possible interactions it can have with external objects, which means that it is a concise specification of what the object does. This makes OOP a flexible system, because an object can be modified or extended with no changes to its external interface. New classes can be added to a system that uses the interfaces of the existing classes.

Objects typically communicate with each other by message passing. A message can send data to an object or request that it invoke a method. The objects can both send and receive messages.

Another key characteristic of OOP is encapsulation, which refers to how the implementation details of a particular class are hidden from all objects outside of the class. Programmers specify what information in an object can be shared with other objects.

A final attribute of object oriented programming languages is polymorphism. Polymorphism means that objects of different types can receive the same message and respond in different ways. The different objects need to have only the same interface (that is, method definition). The calling object (the client) does not need to know exactly what type of object it is calling, only that is has a method of a specific name with defined arguments. Polymorphism is often applied to derived classes, which replace the methods of the parent class with different behaviors. Polymorphism and inheritance together make OOP flexible and easy to extend.

Object-oriented programming proponents claim several large advantages. They maintain that OOP emphasizes modular code that is simple to develop and maintain. OOP is popular in larger software projects, because objects or groups of objects can be divided among teams and developed in parallel. It encourages careful up-front design, which facilitates a disciplined development process. Object-oriented programming seems to provide a more manageable foundation for larger software projects.

The most popular object-oriented programming languages include Java, Visual Basic, C#, C++, and Python.

Other Paradigms

Concurrent programming provides for multiple computations running at once. This often means support for multiple threads of program execution. For example, one thread might

monitor the mouse and keyboard for user input, although another thread performs database accesses. Popular concurrent languages include Ada and Java.

Functional programming languages define subroutines and programs as mathematical functions. These languages are most frequently used in various types of mathematical problem solving. Popular functional languages include LISP and Mathematica.

Event-driven programming is a paradigm where the program flow is determined by user actions. This is in contrast to batch programming, where flow is determined when the program is written. Event-driven programming is used for interactive programs, such as word processors and spreadsheets. The program usually has an event loop, which repeatedly checks for interesting events, such as a keypress or mouse movement. Events cause the execution of trigger functions, which process the events.

Two final paradigms to discuss are compiled languages and interpreted languages. Compiled languages use a program called a compiler to translate source code into machine instructions, usually saved to a separate executable file. Interpreted languages, in contrast, are programs that can be executed from directly from source code by a special program called an interpreter. This distinction refers to the implementation, rather than the language design itself. Most software is compiled, and in theory, any language can be compiled. LISP a well known interpreted language. Some popular implementations, such as Java and C#, use just-in-time compilation. Source programs are compiled into bytecode, which is an executable for an abstract virtual machine. At run time, the bytecode is compiled into native machine code.

Summary

Programming languages are artificial languages designed to control computers and many man hours have been spent to develop new and improved languages. There are thousands of different programming languages, but most conform to a few basic paradigms. Procedural programming languages divide a program's source code into smaller modules. Structured programming languages impose more constraints on program organization and flow, including a ban on GOTO statements. Object-oriented programs organize data structures and code into objects, so that a program becomes a group of cooperating objects. Key features of the object-oriented paradigm are classes, inheritance, data encapsulation, and polymorphism.

References

Books

David Gelernter, Suresh Jagannathan: *Programming Linguistics*, The MIT Press 1990.

Shriram Krishnamurthi: *Programming Languages: Application and Interpretation*. Available online at: http://www.cs.brown.edu/~sk/Publications/Books/ProgLangs/

Websites

Computer Languages History graphical chart

http://www.levenez.com/lang/history.html

History of Computer Languages

http://hopl.murdoch.edu.au/

An Introduction to Programming Languages

http://www.acooke.org/andrew/writing/lang.html